Aditya Mukhopadhyay | Track A - Agentic / Multi-Agent AI

Supplier-delay decision support

Scenario S1: Supply chain risk and optimisation

1. Problem & Domain

A supplier delay matters when usable material runs out before active factory orders can be fulfilled. This POC compares waiting, buying enough to bridge the shortage, and buying enough to protect the safety buffer. It presents extra spending, missed material due dates, ending backlog, buffer depletion and relevant past incidents before a human chooses. Demand comes from remaining active production schedules and the bill of materials (BOM), rather than an unrelated daily-demand constant.

Why Track A: the AI opportunity is coordinating evidence and explaining trade-offs, not predicting a number. Classical ML lacks representative labels here; deterministic rules alone can calculate the choices. Four narrow agent roles add explicit hand-offs and a separate review perspective compared with one long prompt. This is a design rationale, not an experimentally proven advantage over a single LLM.

2. Approach & Algorithm Decisions

Role separation. Disruption verifies the shipment change; Inventory assesses order-derived demand and stock; Procurement recommends among calculated options; Review checks the complete comparison. A deterministic comparison node sits between Inventory and Procurement. Three roles would combine recommendation and review; more roles would add calls without a distinct responsibility. Review uses the same model, so role separation does not imply independent errors.

LangGraph over conversational coordination. Explicit state, conditional edges, interrupt/resume and SQLite checkpoints fit an auditable approval workflow. CrewAI or AutoGen could support looser collaboration, but this fixed sequence is easier to debug within the brief. The trade-off is serial latency rather than parallel agent execution. Typed Pydantic findings carry evidence IDs, status and role fields through previous_results; a snapshot ID binds the comparison.

Code for arithmetic; agents for interpretation. Read-only, parameterized SQLite tools retrieve exact records; a separate local HTTP service supplies shipment notices and offers. SQL is simpler than vector retrieval for keyed operational facts. Code simulates stock/backlog and validates approval, specification, capacity and quote validity. Bounded wait/bridge/buffer enumeration was chosen over MILP because the scope is one material and few offers, not global replenishment optimization. The rule minimizes material due-date misses, ending backlog, spending, then buffer-breach days. This makes service/cost priorities explicit; it claims no global optimum.

Data and 48-hour scope. seed.py creates a deterministic fixture: two materials, two products, three suppliers and five planning days; five synthetic incidents are added separately. These are designed cases, not samples from a fitted distribution. They exercise shortages and invalid offers but cannot establish real supplier reliability. A shared abstract agent shell, TOML prompt skills, reusable tools and dependency injection kept four agents explainable. SQLite, a mock supplier API and hosted Gemini avoid GPU and enterprise-integration setup; real purchasing, authentication and a dashboard were deferred. These are scope choices for the 48-hour brief, not claims of production readiness.

Silent failure controls. Code supplies the comparison; typed findings, key metrics and option IDs are validated, and a supported review is required. Recompute the entire comparison before presentation and after approval; bind case, thread, snapshot and version, persist a 60-minute deadline, and invalidate stale selections. These checks do not prove every free-text claim; a shared model can repeat an error.

3. Results & Error Analysis

Engineering validation: 55/55 local tests pass. They cover calculations, schema guards, bounded revision, missing evidence, provider/source failure, rejection, requested changes, stale selections, expiry, replay prevention and separate-process SQLite resume. They use development fixtures and a scripted runtime: they are not a held-out business benchmark or a 100% agent-accuracy result. No independent held-out evaluation set has yet been collected.

Business example, expected scenario. MAT-01 opens with 100 units and a 20-unit buffer. Active schedules require 40 units/day for five days; a 200-unit receipt shifts from November 3 to November 6, 2026. The comparison is:

Choice	Extra spend	Ending unmet units	Days below buffer
Wait	$0	100	3
Buy 100: bridge	$1,250	0	1
Buy 120: protect buffer	$1,490	0	0

The bridge removes 100 ending unmet units relative to waiting, at $1,250 extra spend; preserving the buffer costs another $240. Under 120% demand, bridge/protect-buffer still end 40/20 units short. A one-day-late replacement leaves zero ending backlog but 20 units miss their material due date. These are calculated what-if scenarios, not observed savings, finished-product delivery guarantees or probabilities. Historical action samples are only 2 wait, 2 bridge and 1 buffer case; they are shown as observations, not causal evidence.

What worked live. Gemini 3.5 Flash-Lite completed all four roles with zero revisions and paused at awaiting_approval; a fresh runner read the checkpoint. Disruption and Inventory actually called tools; Procurement and Review used the supplied deterministic comparison. No live human choice or purchase was submitted. Unchanged approval and terminal failure paths were exercised locally, not established as live-model reliability rates.

What did not work. Gemini 3.8 authenticated and passed a structured smoke check but encountered provider 504 deadlines on inventory work; Flash-Lite was explicitly selected instead of a hidden fallback. Early outputs generated the wrong role or combined success with blocking questions. Application-owned role metadata and clearer field descriptions resolved the observed cases; validators and a single shared correction cycle catch recurrence. These were iterative fixes, not a controlled comparison. Token cost and latency were not benchmarked.

4. Production & Limitations

Production step. Replace the mock with authenticated supplier/ERP adapters and use durable checkpoints with identity, concurrency control and idempotent orders. Monitor failures, stale approvals, tool latency and model usage; pin versions and canary updates with replay tests. The POC caps each agent at six model turns with one SDK retry per request, but four serial roles still add latency and cost.

Limits and remaining evidence.

- The demo uses a small synthetic dataset. It still needs testing against real factory data.
- Production capacity, downtime and lost revenue are not modelled.
- Supplier responses are mocked. Human approval is saved, but no purchase order is sent.
- An independent set of test cases is needed to evaluate performance beyond the development examples.
- Complete agent and tool traces still need to be captured for both successful and failed runs.